Rajalakshmi Engineering College

Name: Keerthana G
Email: 240701254@rajalakshmi.edu.in
Roll no: 240701254
Phone: 9342478755
Branch: REC
Department: CSE - Section 3
Batch: 2028
Degree: B.E - CSE

Scan to verify results

2024_28_III_OOPS Using Java Lab

REC_2028_OOPS using Java_Week 8_MCQ

Attempt : 1
Total Mark : 15
Marks Obtained : 9

Section 1 : MCQ

1. What will be the output for the following code?

```
import java.io.*;

class OutOfStockException extends Exception {
   public OutOfStockException(String message) {
      super(message);
   }
}

class Test {
   public static void main(String[] args) {
      try {
         int stock = 0;
         if (stock == 0) {
            throw new OutOfStockException("Item is out of stock");
```

```
        }
    } catch (OutOfStockException e) {
        System.out.println(e.getMessage());
    }
  }
}
```

Answer

Compilation Error

Status : Wrong ***Marks : 0/1***

2. Which of the following is true about custom exceptions?

Answer

Custom exceptions must extend either Exception or RuntimeException

Status : Correct ***Marks : 1/1***

3. what is the output of the following code?

```
class MyException extends Exception {
  public MyException(String message) {
    super(message);
  }
}

class Test {
  static void check() throws MyException {
    throw new MyException("Custom Exception Occurred");
  }

  public static void main(String[] args) {
    try {
      check();
    } catch (Exception e) {
      System.out.println(e.getMessage());
    }
```

240701254

```
    }
}
```

Answer

Exception in thread "main" java.lang.Exception

Status : Wrong ***Marks : 0/1***

4. What will be the output for the following code?

```
class NegativeBalanceException extends Exception {
    public NegativeBalanceException(String message) {
        super(message);
    }
}

class Test {
    public static void main(String[] args) {
        try {
            double balance = -500;
            if (balance < 0) {
                throw new NegativeBalanceException("Balance cannot be
negative");
            }
        } catch (NegativeBalanceException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

Answer

Error: Balance cannot be negative

Status : Correct ***Marks : 1/1***

5. What will be the output for the following code?

```
import java.io.*;
```

```
class NegativeAgeException extends Exception {
   public NegativeAgeException(String message) {
      super(message);
   }
}

class Test {
   public static void main(String[] args) {
      try {
         int age = -5;
         if (age < 0) {
            throw new NegativeAgeException("Age cannot be negative");
         }
      } catch (NegativeAgeException e) {
         System.out.println(e.getMessage());
      }
   }
}
```

Answer

null

Status : Wrong ***Marks : 0/1***

6. Which keyword is used to explicitly throw a custom exception?

Answer

throws

Status : Wrong ***Marks : 0/1***

7. What will be the output for the following code?

```
class InvalidUsernameException extends Exception {
   public InvalidUsernameException(String message) {
      super(message);
   }
}
```

240701254

```
class Test {
   public static void main(String[] args) {
      try {
         String username = "abc";
         if (username.length() < 5) {
            throw new InvalidUsernameException("Username must be at
least 5 characters long");
         }
      } catch (InvalidUsernameException e) {
         System.out.println(e.getMessage());
      }
   }
}
```

Answer

Username must be at least 5 characters long

Status : Correct ***Marks : 1/1***

8. What will be the output for the following code?

```
import java.io.*;

class UnderageException extends Exception {
   public UnderageException(String message) {
      super(message);
   }
}

class Test {
   public static void main(String[] args) {
      try {
         int age = 17;
         if (age < 18) {
            throw new UnderageException("Underage, cannot proceed");
         }
      } catch (UnderageException e) {
         System.out.println(e.getMessage());
```

240701254

```
    }
  }
}
```

Answer

17

Status : Wrong ***Marks : 0/1***

9. How do you create an unchecked custom exception?

Answer

By extending RuntimeException

Status : Correct ***Marks : 1/1***

10. What is the purpose of a custom exception in Java?

Answer

To create user-defined exceptions for specific scenarios

Status : Correct ***Marks : 1/1***

11. What will happen if a checked custom exception is thrown inside a method without being caught or declared?

Answer

Compilation Error

Status : Correct ***Marks : 1/1***

12. What will be the output of the following code?

```
class MyException extends Exception {
  public MyException() {
    super("Default Exception Message");
  }
```

240701254

```
}

class Test {
    public static void main(String[] args) {
        try {
            throw new MyException();
        } catch (MyException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

Answer

Default Exception Message

Status : Correct ***Marks : 1/1***

13. What will be the output for the following code?

```
import java.io.*;

class TemperatureTooHighException extends Exception {
    public TemperatureTooHighException(String message) {
        super(message);
    }
}

class Test {
    public static void main(String[] args) {
        try {
            int temperature = 110;
            if (temperature > 100) {
                throw new TemperatureTooHighException("Temperature too
high");
            }
        } catch (TemperatureTooHighException e) {
            System.out.println(e.getMessage());
        }
    }
```

240701254

```
}
```

Answer

Temperature too high

Status : Correct ***Marks : 1/1***

14. What will be the output for the following code?

```
class InvalidVotingAgeException extends Exception {
   public InvalidVotingAgeException(String message) {
      super(message);
   }
}

class Test {
   public static void main(String[] args) {
      try {
         int age = 15;
         if (age < 18) {
            throw new InvalidVotingAgeException("You are not eligible to
vote");
         }
         System.out.println("Eligible to vote");
      } catch (InvalidVotingAgeException e) {
         System.out.println(e.getMessage());
      }
   }
}
```

Answer

Exception occurred

Status : Wrong ***Marks : 0/1***

15. what is the output of the following code?

```
class MyException extends Exception {
   public MyException(String message) {
```

```
        super(message);
    }
}

class Test {
    public static void main(String[] args) {
        try {
            throw new MyException("Error occurred");
        } catch (MyException e) {
            System.out.println(e);
        }
    }
}
```

Answer

MyException: Error occurred

Status : Correct ***Marks : 1/1***

240701254

Rajalakshmi Engineering College

Name: Keerthana G
Email: 240701254@rajalakshmi.edu.in
Roll no: 240701254
Phone: 9342478755
Branch: REC
Department: CSE - Section 3
Batch: 2028
Degree: B.E - CSE

Scan to verify results

2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 8_Q1

Attempt : 1
Total Mark : 10
Marks Obtained : 7.5

Section 1 : Coding

1. Problem Statement

Write a program to validate the email address and display suitable exceptions if there is any mistake.

Create 3 custom exception classes as below

DotExceptionAtTheRateExceptionDomainException

A typical email address should have a " . " character, and a "@" character, and also the domain name should be valid. Valid domain names for practice be 'in', 'com', 'net', or 'biz'.

Display Invalid Dot usage, Invalid @ usage, or Invalid Domain message based on email id.

Get the email address from the user, validate the email by checking the

above-mentioned criteria, and print the validity status of the input email address.

Input Format

The first line of input contains the email to be validated.

Output Format

The output prints a Valid email address or an Invalid email address along with the suitable exception

If email ends with . or contains not exactly one . after @, it throws:

DotException: Invalid Dot usage

Invalid email address

If @ appears not exactly once, it throws:

AtTheRateException: Invalid @ usage

Invalid email address

If the part after the last dot is not among accepted domains:

DomainException: Invalid Domain

Invalid email address

If all conditions satisfied then print:

Valid email address

240701254

Refer to the sample input and output for format specifications.

Sample Test Case

Input: sample@gmail.com
Output: Valid email address

Answer

```
import java.util.Scanner;

class DotException extends Exception
{

   DotException(String msg)

{

      super(msg);

}
}

class AtTheRateException extends Exception

{

   AtTheRateException(String msg)

{

      super(msg);
```

```
}
}
class DomainException extends Exception
{

   DomainException(String msg)
{

      super(msg);

}
}
public class Main
{

   public static void main(String[] args)
{

      Scanner sc = new Scanner(System.in);
      String email = sc.nextLine();
      try
{

         validateEmail(email);
         System.out.println("Valid email address");
```

```
} catch (DotException e)
{

        System.out.println("DotException: " + e.getMessage());
        System.out.println("Invalid email address");

} catch (AtTheRateException e)
{

        System.out.println("AtTheRateException: " + e.getMessage());
        System.out.println("Invalid email address");

} catch (DomainException e)
{

        System.out.println("DomainException: " + e.getMessage());
        System.out.println("Invalid email address");

} catch (Exception e)
{

        System.out.println("Invalid email address");

}

}
   static void validateEmail(String email) throws DotException,
```

240701254

```
AtTheRateException, DomainException
{

    if (email.startsWith(".") || email.startsWith("@") || email.endsWith(".") ||
email.endsWith("@") ||
        email.contains("..") || email.contains("@@"))
        throw new DotException("Invalid Dot usage");

    int atCount = email.length() - email.replace("@", "").length();
    if (atCount != 1)
        throw new AtTheRateException("Invalid @ usage");

    int atIndex = email.indexOf('@');
    int lastDot = email.lastIndexOf('.');

    if (lastDot < atIndex + 2 || lastDot == email.length() - 1)
        throw new DotException("Invalid Dot usage");

    String domain = email.substring(lastDot + 1);
    if (!(domain.equals("in") || domain.equals("com") || domain.equals("net") ||
domain.equals("biz")))
        throw new DomainException("Invalid Domain");


}

}
```

Status : Partially correct ***Marks : 7.5/10***

Rajalakshmi Engineering College

Name: Keerthana G
Email: 240701254@rajalakshmi.edu.in
Roll no: 240701254
Phone: 9342478755
Branch: REC
Department: CSE - Section 3
Batch: 2028
Degree: B.E - CSE

Scan to verify results

2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 8_Q2

Attempt : 1
Total Mark : 10
Marks Obtained : 10

Section 1 : Coding

1. Problem Statement

Elsa, a busy professional, is using a scheduling application to plan her meetings efficiently. The application requires users to input meeting durations in minutes, ensuring that the duration is a positive integer and does not exceed 240 minutes (4 hours). Elsa needs a program to assist her in scheduling meetings securely with proper exception handling.

Create a Java class named ElsaMeetingScheduler.Implement a custom exception:InvalidDurationException for invalid meeting duration entries.Implement the main method to interactively take user input for a meeting duration.Implement the validateMeetingDuration method to validate the meeting duration based on the specified rules and throw a custom exception if the validation fails.Print appropriate success or error messages based on the meeting duration.

Implement a custom exception, InvalidDurationException, to handle cases where the entered meeting duration does not meet the specified criteria.

Input Format

The input consists of an integer value 'n', representing the meeting duration.

Output Format

The output is displayed in the following format:

If the entered meeting duration meets the specified criteria, the program outputs

"Meeting scheduled successfully!"

If the entered meeting duration is invalid, the program outputs an error message indicating the issue.

"Error: Invalid meeting duration. Please enter a positive integer not exceeding 240 minutes (4 hours)."

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 120
Output: Meeting scheduled successfully!

Answer

```
import java.util.Scanner;

class InvalidDurationException extends Exception

{


   InvalidDurationException(String msg)

{
```

240701254

```
        super(msg);

    }

    }

    class ElsaMeetingScheduler

    {


       public static void main(String[] args)

    {


        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        try

    {


            validateMeetingDuration(n);
            System.out.println("Meeting scheduled successfully!");


    } catch (InvalidDurationException e)

    {


            System.out.println("Error: Invalid meeting duration. Please enter a positive
    integer not exceeding 240 minutes (4 hours).");


    }


    }
```

240701254

```
    static void validateMeetingDuration(int n) throws InvalidDurationException
{
        if (n <= 0 || n > 240)
{
            throw new InvalidDurationException("Invalid meeting duration");
}
}
}
```

Status : Correct **Marks : 10/10**

Rajalakshmi Engineering College

Name: Keerthana G
Email: 240701254@rajalakshmi.edu.in
Roll no: 240701254
Phone: 9342478755
Branch: REC
Department: CSE - Section 3
Batch: 2028
Degree: B.E - CSE

Scan to verify results

2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 8_Q4

Attempt : 1
Total Mark : 10
Marks Obtained : 10

Section 1 : Coding

1. Problem Statement

A local municipality is implementing an online voting system for a community event and wants to ensure that only eligible voters (those aged 18 or older) can participate.

Your task is to develop a program that validates the age of individuals attempting to vote online. If the user's age is below 18, the program should throw a custom exception, InvalidAgeException, preventing them from casting their vote. If the input is invalid, catch the appropriate InputMismatchException and print the in-built exception message.

Input Format

The input consists of an integer representing the age.

Output Format

240701254

If the age is 18 or older, print "Eligible to vote"

If the age is below 18, print "Exception occurred: InvalidAgeException: Age is not valid to vote"

If there is any other type of exception, print "An error occurred: " followed by the in-built exception message.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 20
Output: Eligible to vote

Answer

```
// You are using Java
import java.util.*;

class InvalidAgeException extends Exception

{


   InvalidAgeException(String msg)

{


      super(msg);


}

}

public class Main

{
```

240701254

```
public static void main(String[] args)
{

    Scanner sc = new Scanner(System.in);
    try
{

        int age = sc.nextInt();
        validateAge(age);
        System.out.println("Eligible to vote");

} catch (InvalidAgeException e)
{

        System.out.println("Exception occurred: InvalidAgeException: Age is not
valid to vote");

} catch (InputMismatchException e)
{

        System.out.println("An error occurred: " + e);

} catch (Exception e)
{

        System.out.println("An error occurred: " + e);
```

```
}
}
    static void validateAge(int age) throws InvalidAgeException
{

        if (age < 18)
{
            throw new InvalidAgeException("Age is not valid to vote");
}

}
}
```

Status : Correct ***Marks : 10/10***

Rajalakshmi Engineering College

Name: Keerthana G
Email: 240701254@rajalakshmi.edu.in
Roll no: 240701254
Phone: 9342478755
Branch: REC
Department: CSE - Section 3
Batch: 2028
Degree: B.E - CSE

Scan to verify results

2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 8_Q5

Attempt : 1
Total Mark : 10
Marks Obtained : 10

Section 1 : Coding

1. Problem Statement

In a file management system, users are required to provide a valid file name when creating new files. The system enforces specific rules for file names to maintain consistency and avoid potential issues. Your task is to implement a Java program named FileNameValidator that takes user input for a file name and validates it according to the specified rules.

Rules for Valid File Name:

The file name must consist of alphanumeric characters (letters and digits) only.The file name must have a minimum length of 3 characters.

Implement a custom exception, FileNameValidator, to handle cases where the entered filename does not meet the specified criteria.

Input Format

The input consists of a string S, representing the desired filename.

Output Format

The output is displayed in the following format:

If the entered file name meets the specified criteria, the program outputs

"Valid file name"

If the entered file name does not meet the criteria and triggers the InvalidFileNameException, the program outputs

"Error: Invalid file name. It must be alphanumeric and have a minimum length of 3 characters."

Refer to the sample output for formatting specifications.

Sample Test Case

Input: myfile123
Output: Valid file name

Answer

```
// You are using Java
import java.util.Scanner;

class InvalidFileNameException extends Exception

{


   InvalidFileNameException(String msg)

{


      super(msg);


}
```

240701254

```
}
class Main
{

    public static void main(String[] args)
{

        Scanner sc = new Scanner(System.in);
        String fileName = sc.nextLine();
        try
{

            validateFileName(fileName);
            System.out.println("Valid file name");

} catch (InvalidFileNameException e)
{

            System.out.println("Error: Invalid file name. It must be alphanumeric and
have a minimum length of 3 characters.");

}

}

    static void validateFileName(String fileName) throws
InvalidFileNameException
{
```

240701254

```
        if (fileName.length() < 3 || !fileName.matches("[A-Za-z0-9]+"))
    {
            throw new InvalidFileNameException("Invalid file name");
    }
    }
}
```

Status : Correct ***Marks : 10/10***

Rajalakshmi Engineering College

Name: Keerthana G
Email: 240701254@rajalakshmi.edu.in
Roll no: 240701254
Phone: 9342478755
Branch: REC
Department: CSE - Section 3
Batch: 2028
Degree: B.E - CSE

Scan to verify results

2024_28_III_OOPS Using Java Lab

REC_2028_OOPS using Java_Week 8_CY

Attempt : 1
Total Mark : 40
Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

Tim was tasked with creating a user profile system that validates the user's date of birth input. The system should throw a custom exception, InvalidDateOfBirthException, if the date is not in the specified format "dd-mm-yyyy" or if it represents an invalid calendar date.

The main method takes user input, validates the date of birth, and prints whether it is valid or not.

Input Format

The input consists of a string, representing the date of birth of the user.

Output Format

The output displays one of the following results:

If the entered date of birth is valid according to the specified format, the program prints:

"[Date] is a valid date of birth"

If the entered date of birth is not valid according to the specified format, the program prints:

"Invalid date: [Date]"

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 01-01-2000
Output: 01-01-2000 is a valid date of birth

Answer

```
// You are using Java
import java.util.Scanner;
import java.text.SimpleDateFormat;
import java.text.ParseException;

class InvalidDateOfBirthException extends Exception
{

   InvalidDateOfBirthException(String msg)

{

      super(msg);

}
}
```

240701254

```
public class Main
{

    public static void main(String[] args)
{

        Scanner sc = new Scanner(System.in);
        String dob = sc.nextLine();
        try
{

            validateDOB(dob);
            System.out.println(dob + " is a valid date of birth");

} catch (InvalidDateOfBirthException e)
{

            System.out.println("Invalid date: " + dob);

}

}

    static void validateDOB(String dob) throws InvalidDateOfBirthException
{

        SimpleDateFormat sdf = new SimpleDateFormat("dd-MM-yyyy");
        sdf.setLenient(false);
```

240701254

```
    try
{
        sdf.parse(dob);

} catch (ParseException e)
{

        throw new InvalidDateOfBirthException("Invalid date");

}

}

}
```

Status : Correct ***Marks : 10/10***

2. Problem Statement

Alice is designing a program that requires users to enter positive numbers. She wants to implement a solution that validates whether the entered number is positive. In case the input is not a positive number, she wants to throw a custom exception.

The number should be a positive integer.If this condition is violated, the program should throw a custom exception:InvalidPositiveNumberException with the message "Invalid input. Please enter a positive integer."

Implement a custom exception, InvalidPositiveNumberException , to handle cases where the entered number does not meet the specified criteria.

240701254

Input Format

The input consists of an integer value 'n', representing the entered number.

Output Format

The output is displayed in the following format:

If the validation passes, print

"Number {number} is positive."

The {number} represents the entered positive integer.

If the entered number is negative then it displays

"Error: Invalid input. Please enter a positive integer."

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 100
Output: Number 100 is positive.

Answer

```
// You are using Java
import java.util.Scanner;

class InvalidPositiveNumberException extends Exception

{


   InvalidPositiveNumberException(String msg)

{


      super(msg);
```

240701254

```
}
}

public class Main

{


    public static void main(String[] args)

{


        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        try

{


            validatePositiveNumber(n);
            System.out.println("Number " + n + " is positive.");


} catch (InvalidPositiveNumberException e)

{


            System.out.println("Error: " + e.getMessage());


}


}

    static void validatePositiveNumber(int n) throws
InvalidPositiveNumberException
```

```
{
        if (n <= 0)
{
            throw new InvalidPositiveNumberException("Invalid input. Please enter a positive integer.");
}
}
}
```

Status : Correct ***Marks : 10/10***

3. Problem Statement

Hemanth is designing a banking system for XYZ Bank. The system should allow customers to perform deposit, withdrawal, and balance inquiry operations. Implement exception handling for scenarios involving invalid transaction amounts or insufficient funds.

Create two custom exception classes, InvalidAmountException and InsufficientFundsException, both extending the Exception class.Throw an InvalidAmountException with a message if the deposit amount is less than or equal to zero.Throw an InsufficientFundsException if the withdrawal amount is greater than the available balance.Deduct the withdrawal amount from the balance if the withdrawal is successful.

Assist Hemanth in designing the program.

Input Format

The first line of input consists of a double value B, representing the initial balance.

240701254

The second line consists of a double value D, representing the deposit amount.

The third line consists of a double value W, representing the withdrawal amount.

Output Format

If the withdrawal is successful, print the amount withdrawn and the current balance, rounded off to one decimal place.

If an InvalidAmountException occurs, print "Error: [D] is not valid".

If an InsufficientFundsException occurs, print "Error: Insufficient funds".

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 1050.1
270.2
150.3

Output: Amount Withdrawn: 150.3
Current Balance: 1170.0

Answer

```
// You are using Java
import java.util.Scanner;

class InvalidAmountException extends Exception

{


   InvalidAmountException(String msg)

{


      super(msg);
```

240701254

```
}
}
class InsufficientFundsException extends Exception
{

    InsufficientFundsException(String msg)
{

        super(msg);

}
}
class BankAccount
{

    double balance;
    BankAccount(double balance)
{

        this.balance = balance;

}
    void deposit(double amount) throws InvalidAmountException
{
```

240701254

```
        if (amount <= 0)
    {

            throw new InvalidAmountException(amount + " is not valid");

    }
        balance += amount;

    }
      void withdraw(double amount) throws InsufficientFundsException
    {

        if (amount > balance)

    {

            throw new InsufficientFundsException("Insufficient funds");

    }
        balance -= amount;
        System.out.printf("Amount Withdrawn: %.1f%n", amount);
        System.out.printf("Current Balance: %.1f%n", balance);

    }

    }

    public class Main

    {
```

240701254

```
public static void main(String[] args)
{

    Scanner sc = new Scanner(System.in);
    double B = sc.nextDouble();
    double D = sc.nextDouble();
    double W = sc.nextDouble();

    BankAccount account = new BankAccount(B);

    try
{

        account.deposit(D);
        account.withdraw(W);


} catch (InvalidAmountException e)

{

        System.out.println("Error: " + e.getMessage());

} catch (InsufficientFundsException e)

{


        System.out.println("Error: " + e.getMessage());


}

}
```

240701254

}

Status : Correct ***Marks : 10/10***

4. Problem Statement

A company is developing a user registration system that requires users to provide valid email addresses. The development team is implementing an EmailValidator program to ensure that the entered email addresses meet certain criteria using exception handling.

The email address must contain the "@" symbol.The email address must consist of a non-empty username(before "@" symbol) and a non-empty domain(after "'@" symbol).The domain part of the email address must contain at least one period (".").The email address must not contain leading or trailing spaces.

Implement a custom exception, InvalidEmailException, to fulfill the company's requirements and validate it according to the specified rules.

Input Format

The input consists of a string value 's', which represents the email address.

Output Format

The output is displayed in the following format:

If the entered email address is valid according to the specified rules, the program prints:

"Email address is valid!"

If the entered email address misses the username or domain part or misses "@" symbol or has two or more "@" symbols or misses '.' in the domain part it outputs:

"Error: Invalid email format."

Refer to the sample output for formatting specifications.

Sample Test Case

Input: johndoe@example.com
Output: Email address is valid!

Answer

```
// You are using Java
import java.util.Scanner;

class InvalidEmailException extends Exception

{

   InvalidEmailException(String msg)

{

      super(msg);

}

}
public class Main

{

   public static void main(String[] args)

{

      Scanner sc = new Scanner(System.in);
      String email = sc.nextLine();
      try
```

240701254

```
{

        validateEmail(email);
        System.out.println("Email address is valid!");

} catch (InvalidEmailException e)

{

        System.out.println("Error: Invalid email format.");

}

}

   static void validateEmail(String email) throws InvalidEmailException

{

      if (email.startsWith(" ") || email.endsWith(" "))

{

        throw new InvalidEmailException("Invalid email");

}
      int atCount = 0;
      for (char c : email.toCharArray())

{

        if (c == '@') atCount++;
```

240701254

```
}
        if (atCount != 1)
{

            throw new InvalidEmailException("Invalid email");

}
        String[] parts = email.split("@");
        if (parts.length != 2 || parts[0].isEmpty() || parts[1].isEmpty() || !
parts[1].contains("."))
{

            throw new InvalidEmailException("Invalid email");

}

}

}
```

Status : Correct ***Marks : 10/10***

Rajalakshmi Engineering College

Name: Keerthana G
Email: 240701254@rajalakshmi.edu.in
Roll no: 240701254
Phone: 9342478755
Branch: REC
Department: CSE - Section 3
Batch: 2028
Degree: B.E - CSE

Scan to verify results

2024_28_III_OOPS Using Java Lab

REC_2028_OOPS using Java_Week 8_PAH

Attempt : 1
Total Mark : 40
Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

Daniel is developing a program to verify the age of users. He wants to ensure that the entered age is within a valid range. Write a program to help Daniel implement this age-checking feature using custom exceptions.

Daniel needs a program that takes an integer input representing a person's age.If the age is between 0 and 150 (inclusive), the program should print "Age is valid!".If the age is less than 0 or greater than 150, the program should throw a custom exception (InvalidAgeException) with the message "Invalid age. Please enter an age between 0 and 150."

Implement a custom exception, InvalidAgeException, to handle cases where the entered age does not meet the specified criteria.

Input Format

240701254

The input consists of an integer value 'n', representing the age.

Output Format

The output is displayed in the following format:

If the age is valid (between 0 and 150, inclusive), print

"Age is valid!".

If the age is invalid, print

"Error: Invalid age. Please enter an age between 0 and 150."

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 45
Output: Age is valid!

Answer

```
// You are using Java
import java.util.Scanner;

class InvalidAgeException extends Exception
{

   InvalidAgeException(String msg)

{

      super(msg);

}
}
```

240701254

```
public class Main
{

    public static void main(String[] args)
{

        Scanner sc = new Scanner(System.in);
        int age = sc.nextInt();
        try
{

            validateAge(age);
            System.out.println("Age is valid!");

} catch (InvalidAgeException e)
{

            System.out.println("Error: Invalid age. Please enter an age between 0 and
150.");

}

}

    static void validateAge(int age) throws InvalidAgeException
{

        if (age < 0 || age > 150)
```

240701254

```
{
        throw new InvalidAgeException("Invalid age. Please enter an age between 0 and 150.");

}

}

}
```

Status : Correct ***Marks : 10/10***

2. Problem Statement

You are tasked to create a program that defines a custom exception GradeException. The program should include a Student class with fields for the student's name, age, and grade. Implement a method in the Student class that checks the grade, and if the grade is below 40, it should throw a GradeException. Otherwise, it should display the student's details.

Input Format

The input consists of three parameters in separate lines:

1. A string representing the student's name.
2. An integer representing the student's age.
3. An integer representing the student's grade.

Output Format

The output will display the student's details if the grade is valid.

If the grade is below 40, the program will display an error message "Grade is below 40".

240701254

Refer to the sample output for formatting specifications.

Sample Test Case

Input: Alice
20
85

Output: Name: Alice
Age: 20
Grade: 85

Answer

```
// You are using Java
import java.util.Scanner;

class GradeException extends Exception

{


    GradeException(String msg)

{


        super(msg);


}

}

class Student

{


    String name;
    int age;
    int grade;

    Student(String name, int age, int grade)
```

240701254

```
{

        this.name = name;
        this.age = age;
        this.grade = grade;


}

    void checkGrade() throws GradeException

{

        if (grade < 40)

{


            throw new GradeException("Grade is below 40");


} else

{


            System.out.println("Name: " + name);
            System.out.println("Age: " + age);
            System.out.println("Grade: " + grade);


}


}

}
public class Main
```

240701254

```
{

    public static void main(String[] args)

{

        Scanner sc = new Scanner(System.in);
        String name = sc.nextLine();
        int age = sc.nextInt();
        int grade = sc.nextInt();

        Student s = new Student(name, age, grade);
        try

{

            s.checkGrade();

} catch (GradeException e)

{

            System.out.println(e.getMessage());

}

}

}
```

Status : Correct ***Marks : 10/10***

3. Problem Statement

An HR software system is being developed to process employee payrolls. During payroll processing, the system must ensure that no employee has a negative salary and that no employee's salary exceeds 2,00,000. If either condition occurs, the system should throw a custom exception.

Create a custom exception InvalidSalaryException and a class Employee that processes salary according to the following rules:

If salary < 0, throw InvalidSalaryException with the message: "Salary cannot be negative". If salary > 200000, throw InvalidSalaryException with the message: "Salary exceeds threshold limit". Otherwise, display: "Salary processed successfully for <empName>: <salary>".

The payroll processing should always display: "Payroll process completed" at the end, regardless of whether an exception occurs.

Input Format

The first line of input contains an integer representing the employee ID.

The second line contains a string representing the employee's name.

The third line contains a floating-point number representing the salary of the employee.

Output Format

If the salary is valid: "Salary processed successfully for <empName>: <salary>"

"Payroll process completed"

If the salary is invalid: "<Exception Message>"

"Payroll process completed"

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 101

240701254

Rahul
150000.0
Output: Salary processed successfully for Rahul: 150000.0
Payroll process completed

Answer

```
// You are using Java
import java.util.Scanner;

class InvalidSalaryException extends Exception

{


    InvalidSalaryException(String msg)

{


        super(msg);


}

}

class Employee
{


    int empId;
    String empName;
    double salary;

    Employee(int empId, String empName, double salary)

{


        this.empId = empId;
        this.empName = empName;
```

240701254

```
        this.salary = salary;

    }

    void processSalary() throws InvalidSalaryException

    {

        if (salary < 0)

    {

            throw new InvalidSalaryException("Salary cannot be negative");

    } else if (salary > 200000)

    {

            throw new InvalidSalaryException("Salary exceeds threshold limit");

    } else
    {

            System.out.println("Salary processed successfully for " + empName + ": "
    + salary);

    }

    }

    }
```

240701254

```
public class Main
{
    public static void main(String[] args)
{
        Scanner sc = new Scanner(System.in);
        int empId = sc.nextInt();
        sc.nextLine(); // consume newline
        String empName = sc.nextLine();
        double salary = sc.nextDouble();

        Employee emp = new Employee(empId, empName, salary);
        try
{
            emp.processSalary();
} catch (InvalidSalaryException e)
{
            System.out.println(e.getMessage());
} finally
{
            System.out.println("Payroll process completed");
}
```

240701254

}

}

Status : Correct ***Marks : 10/10***

4. Problem Statement

Enigma is developing a simple web application that takes a user-input URL, validates it, and throws a custom exception InvalidURLFormatException if the URL does not start with "http://" or "https://".

The main method prompts the user for input, validates the URL, and prints whether it is valid or not.

Input Format

The input consists of a string, representing the URL entered by the user.

Output Format

The output displays one of the following results:

If the entered URL is valid according to the specified format, the program prints:

"[URL] is a valid URL"

If the entered URL is not valid according to the specified format, the program prints:

"Invalid URL format: [URL]"

Refer to the sample output for formatting specifications.

240701254

Sample Test Case

Input: http://www.example.com
Output: http://www.example.com is a valid URL

Answer

```
// You are using Java
import java.util.Scanner;

class InvalidURLFormatException extends Exception

{


   InvalidURLFormatException(String msg)

{


      super(msg);


}

}

public class Main

{


   public static void main(String[] args)

{


      Scanner sc = new Scanner(System.in);
      String url = sc.nextLine();
      try

{
```

240701254

```
            validateURL(url);
            System.out.println(url + " is a valid URL");

    } catch (InvalidURLFormatException e)

    {

            System.out.println("Invalid URL format: " + url);

    }

    }

    static void validateURL(String url) throws InvalidURLFormatException

    {

        if (!(url.startsWith("http://") || url.startsWith("https://")))

    {

            throw new InvalidURLFormatException("Invalid URL format");

    }

    }

    }
```

Status : Correct ***Marks : 10/10***